DevOps for AI - Practical Exam

Statement-wise command sheet for all 12 problems (Git, GitHub Desktop, Jenkins, Docker, Compose, Minikube, CI/CD)

How to read: `grey monospace = type this in git-bash;` *italic = what to click / what to expect. Left column = statement letter from the question paper.*

One-Time Setup (do this before the exam)

Replace <username> with your GitHub username everywhere. For Problems 1-4 create an EMPTY repository on github.com first (no README, no .gitignore, no licence). Commands are for git-bash on Windows.

Identity	`git config --global user.name "Your Name"`
	`git config --global user.email "you@example.com"`
	`git --version`

Login	*GitHub does not accept account passwords in git-bash. Either sign in through the browser pop-up (Git Credential Manager) or use a Personal Access Token (GitHub > Settings > Developer settings > Tokens (classic) > scope: repo) when asked for a password.*
Branch name	*Exam says master/main. This sheet uses main. If your Git creates master, rename with: git branch -M main*
Pager	*If git diff / git log opens a screen with (END) or a colon, press q to exit.*

> **EXAM TIP: after every lettered step take a screenshot - the command output (git log, git status, git branch -a) AND the GitHub page. Examiners check the proof, not just the final state.**

Problem 1 - Git-bash: multiplication table + branch GIT-EXAM

a	*Create the folder and initialise Git*
	`mkdir ds-project`
	`cd ds-project`
	`git init`
	`git branch -M main`

b	*Create an empty repo named ds-project on GitHub, then link and push*
	`echo "# Data Science Project" > README.md`
	`git add README.md`
	`git commit -m "Initial commit"`
	`git remote add origin https://github.com/<username>/ds-project.git`
	`git push -u origin main`

c	`cat > program1.py << 'EOF'`
	`n = int(input("Enter a number: "))`
	`for i in range(1, 11):`
	`    print(f"{n} x {i} = {n * i}")`
	`EOF`
	`python program1.py`

d	`git status`
	`git add program1.py`
	`git commit -m "Add program1: multiplication table"`
	`git push origin main`
	`git log --oneline`
	Check remote: refresh the GitHub repo page - program1.py and the commit message are visible.

e	`git checkout -b GIT-EXAM` `cat > program2.py << 'EOF'`

	``
	a = int(input("Enter first number: "))
	b = int(input("Enter second number: "))
	if a >= b:
	    print("Greatest number is:", a)
	else:
	    print("Greatest number is:", b)
	EOF
	python program2.py
	``

f	``` git add program2.py git commit -m "Add program2: greatest of two numbers" git push -u origin GIT-EXAM git branch -a git diff main GIT-EXAM git diff --name-status main GIT-EXAM git log --oneline --graph --all git checkout main ls ``` *program2.py is missing on main but present on GIT-EXAM. On GitHub use the branch dropdown, or open github.com/<username>/ds-project/compare/main...GIT-EXAM*

Problem 2 - Git-bash: even/odd, reverse string, pull request

a	``` mkdir ds-project2 cd ds-project2 git init git branch -M main ```
b	*Create an empty repo ds-project2 on GitHub first* ``` echo "# Data Science Project" > README.md git add README.md git commit -m "Initial commit" git remote add origin https://github.com/<username>/ds-project2.git git push -u origin main ```

c	```
	cat > program1.py << 'EOF'
	n = int(input("Enter a number: "))
	if n % 2 == 0:
	    print(n, "is Even")
	else:
	    print(n, "is Odd")
	EOF
	python program1.py
	``

d	``` git add program1.py git commit -m "Add program1: even odd check" git push origin main git log --oneline ```
e	``` git checkout -b GIT-EXAM cat > program2.py << 'EOF' s = input("Enter a string: ") print("Reversed string:", s[::-1]) EOF python program2.py ```
f	``` git add program2.py git commit -m "Add program2: reverse a string" git push -u origin GIT-EXAM git diff main GIT-EXAM git log --oneline --graph --all ```
g	*GitHub web: open the repo > click Compare & pull request > base: main <- compare: GIT-EXAM > Create pull request > Merge pull request > Confirm merge.* *Or with GitHub CLI:* ``` gh pr create --base main --head GIT-EXAM --title "Add reverse string program" --body "Merge GIT-EXAM into main" ```

	`gh pr merge --merge`
	Sync local copy and show that main now has program2.py:
	`git checkout main`
	`git pull origin main`
	`ls`

Problem 3 - Git-bash: two branches, pull, conflict, PR between branches

a	`mkdir ds-project3`
	`cd ds-project3`
	`git init`
	`git branch -M main`

b	*Create an empty repo ds-project3 on GitHub first*
	`echo "# Data Science Project" > README.md`
	`git add README.md`
	`git commit -m "Initial commit"`
	`git remote add origin https://github.com/<username>/ds-project3.git`
	`git push -u origin main`

c	`git checkout -b GIT-EXAM-1`
	`cat > program2.py << 'EOF'`
	`s = input("Enter a string: ")`
	`if s == s[::-1]:`
	`    print("Palindrome")`
	`else:`
	`    print("Not a palindrome")`
	`EOF`

d	`cat > program1.py << 'EOF'`
	`for i in range(1, 11):`
	`    print(f"Square of {i} = {i * i}")`
	`EOF`
	`python program1.py`

e	`git add .`
	`git commit -m "Add program1 (squares) and program2 (palindrome)"`
	`git push -u origin GIT-EXAM-1`

f	*Create GIT-EXAM-2 from main, so the pull in step g really brings new files in.*
	`git checkout main`
	`git checkout -b GIT-EXAM-2`

g	`git pull origin GIT-EXAM-1`
	`ls`
	program1.py and program2.py now appear in GIT-EXAM-2.

h	*program2.py already exists after the pull, so overwrite it with different palindrome logic.*
	`cat > program2.py << 'EOF'`
	`s = input("Enter a string: ").lower()`
	`rev = ""`
	`for ch in s:`
	`    rev = ch + rev`
	`print("Palindrome" if s == rev else "Not a palindrome")`
	`EOF`

i	*Modify program1.py in GIT-EXAM-2 (changed print line):*
	`cat > program1.py << 'EOF'`
	`for i in range(1, 11):`
	`    print("Square of", i, "is", i ** 2)`
	`EOF`

j	`git add .`
	`git commit -m "Modify program1 and rewrite program2 in GIT-EXAM-2"`
	`git push -u origin GIT-EXAM-2`

	To make a REAL conflict, change the same line differently on GIT-EXAM-1:
	`git checkout GIT-EXAM-1`
	`cat > program1.py << 'EOF'`
	`for i in range(1, 11):`
	`    print(i, "squared =", i * i)`
	`EOF`
	`git commit -am "Change print format in GIT-EXAM-1"`
	`git push origin GIT-EXAM-1`
	Demonstrate differences:
	`git diff main GIT-EXAM-1`
	`git diff GIT-EXAM-1 GIT-EXAM-2`
	`git log --oneline --graph --all`

k	*GitHub: Pull requests > New pull request > base: GIT-EXAM-1 <- compare: GIT-EXAM-2 > Create. GitHub shows "This branch has conflicts that must be resolved". Resolve locally:*
	`git checkout GIT-EXAM-2`
	`git merge origin/GIT-EXAM-1`
	Git prints CONFLICT (content) in program1.py. Open the file, delete the <<<<<<<, ======= and >>>>>>> lines, keep the code you want, save. Then:
	`git add program1.py`
	`git commit -m "Resolve merge conflict in program1.py"`
	`git push origin GIT-EXAM-2`
	Refresh the PR: it now says it can be merged. Click Merge pull request > Confirm merge.

Problem 4 - Git-bash: .gitignore for the IGNORE directory

a	`mkdir ds-project4`
	`cd ds-project4`
	`git init`
	`git branch -M main`

b	*Create an empty repo ds-project4 on GitHub first*
	`echo "# Data Science Project" > README.md`
	`git add README.md`
	`git commit -m "Initial commit"`
	`git remote add origin https://github.com/<username>/ds-project4.git`
	`git push -u origin main`

c	`git checkout -b GIT-EXAM-1`
	`cat > program2.py << 'EOF'`
	`s = input("Enter a string: ")`
	`print("Palindrome" if s == s[::-1] else "Not a palindrome")`
	`EOF`

d	`cat > program1.py << 'EOF'`
	`for i in range(1, 11):`
	`    print(f"Square of {i} = {i * i}")`
	`EOF`

e	`git add .`
	`git commit -m "Add program1 and program2"`
	`git push -u origin GIT-EXAM-1`

f	`mkdir IGNORE`
	`echo "test file 1" > IGNORE/test1.txt`
	`echo "test file 2" > IGNORE/test2.txt`
	`git status`
	git status shows IGNORE/ as untracked - screenshot this (before).

g	`echo "IGNORE/" > .gitignore`
	`git status`
	Now IGNORE/ disappears from the list and only .gitignore is untracked (after).

h	`git add .`

	`git commit -m "Add .gitignore to ignore IGNORE directory"`
	`git push origin GIT-EXAM-1`
	`git check-ignore -v IGNORE/test1.txt`
	`git ls-files`
	git ls-files does not list IGNORE/. On GitHub (branch GIT-EXAM-1) the IGNORE folder is absent but .gitignore is present.

NOTE: .gitignore only works for files that are NOT already tracked. Create it before running git add on those files.

Problem 5 - GitHub Desktop: same task as Problem 4 (GUI)

a	*File > New repository... > Name: ds-project-gui > Local path: choose a folder > tick "Initialize this repository with a README" > Git ignore: Python (optional) > Create repository.*
b	*Click Publish repository (top bar) > keep the name > untick "Keep this code private" > Publish repository. Verify: Repository > View on GitHub.*

c	*Branch > New branch... (Ctrl+Shift+N) > name: GIT-EXAM-1 > Create branch.*
	Repository > Show in Explorer (Ctrl+Shift+F) > create program2.py (or Repository > Open in Visual Studio Code) and write:
	`s = input("Enter a string: ")`
	`print("Palindrome" if s == s[::-1] else "Not a palindrome")`

d	*Create program1.py in the same folder:*
	`for i in range(1, 11):`
	`    print(f"Square of {i} = {i * i}")`

e	*Changes tab > tick both files > Summary: "Add program1 and program2" > Commit to GIT-EXAM-1 > click Publish branch (first time) / Push origin.* *Verify: Repository > View on GitHub > switch to branch GIT-EXAM-1.*
f	*Repository > Show in Explorer > New folder IGNORE > inside it create test1.txt and test2.txt (type some text). GitHub Desktop lists them under Changes - screenshot (before).*
g	*Repository > Repository settings... > Ignored files > type IGNORE/ > Save. This creates .gitignore. (Alternative: right-click a file in Changes > Ignore folder.) The IGNORE files vanish from Changes - screenshot (after).*
h	*Tick .gitignore > Summary: "Ignore IGNORE folder" > Commit to GIT-EXAM-1 > Push origin.* *Verify: Repository > View on GitHub: .gitignore is present, the IGNORE folder is NOT.*

Problem 6 - Git-bash: clone, fork, feature branch, pull request

a	`git clone https://github.com/KalyanM45/AI-ProjectGallery.git`
	`cd AI-ProjectGallery`
	`git branch`
	`git remote -v`

b	*A branch can be created locally, but pushing it is denied because you have no write access to someone else's repo:*
	`git checkout -b test-branch`
	`echo "test" > test.txt`
	`git add test.txt`
	`git commit -m "test commit"`
	`git push -u origin test-branch`
	Expected: "Permission to KalyanM45/AI-ProjectGallery.git denied to <username>" / 403. Screenshot it

	as proof, then clean up:
	`git checkout -`
	`git branch -D test-branch`

c	*On GitHub open the repo > Fork (top right) > Create fork. Or with GitHub CLI:* `gh repo fork KalyanM45/AI-ProjectGallery --remote=false`

d	*Point your local copy to YOUR fork, keep the original as upstream:*
	`git remote rename origin upstream`
	`git remote add origin https://github.com/<username>/AI-ProjectGallery.git`
	`git remote -v`
	`git checkout -b GIT-EXAM`

e	`mkdir my-contribution`
	`echo "# My AI project contribution" > my-contribution/README.md`
	`echo "print('Hello from my contribution')" > my-contribution/demo.py`
	`git add .`
	`git commit -m "Add my contribution"`

f	`git push -u origin GIT-EXAM` *On your fork: click Compare & pull request > base repository: KalyanM45/AI-ProjectGallery, base: main <- head: <username>:GIT-EXAM > add title and description > Create pull request.*

g	*If the PR shows conflicts, or the reviewer asks for changes (use the repo's default branch name if it is not main):*
	`git fetch upstream`
	`git merge upstream/main`
	On CONFLICT: open the file, remove the <<<<<<<, =======, >>>>>>> markers, keep the correct code, then:
	`git add .`
	`git commit -m "Resolve merge conflicts with upstream"`
	`git push origin GIT-EXAM`
	Reviewer feedback: edit the files, then git add . > git commit -m "Address review feedback" > git push origin GIT-EXAM. The open PR updates automatically.

Problem 7 - GitHub Desktop: clone, fork, feature branch, pull request

a	*File > Clone repository... > URL tab > https://github.com/KalyanM45/Spam-EmailDetection.git > choose a local path > Clone.*
b	*Branch > New branch... > name: test-branch > Create. Add any small file, Commit to test-branch, then click Publish branch.* *GitHub Desktop shows that you do not have write access (offers to fork) or a 403 / permission denied error. Screenshot this as proof that creating/pushing a branch is disallowed.*
c	*In that dialog click Create a Fork > choose "To contribute to the parent project" > Continue. (Or fork on github.com first and clone your fork.) Desktop now points origin to your fork and keeps the original as upstream.*
d	*Branch > New branch... > name: GIT-EXAM > Create branch (based on the default branch).*
e	*Repository > Show in Explorer > add or edit a file (for example my_contribution.py) > Changes tab > Summary: "Add my contribution" > Commit to GIT-EXAM.*
f	*Click Publish branch > Branch > Create pull request (Ctrl+R). The browser opens: confirm base repository KalyanM45/Spam-EmailDetection <- your fork's GIT-EXAM > Create pull request.*
g	*Branch > Update from KalyanM45/Spam-EmailDetection/main. If conflicts appear: Open in Visual Studio Code > remove the conflict markers > save > Mark as resolved > Continue merge > Commit*

	merge > Push origin. *For reviewer feedback: edit the file > Commit to GIT-EXAM > Push origin. The PR updates automatically.*

Problem 8 - Jenkins: Pipeline from SCM with auto-trigger

a

```
mkdir jenkins-demo
cd jenkins-demo
git init
git branch -M main
echo "print('Hello from Jenkins')" > app.py
echo "flask" > requirements.txt
git add .
git commit -m "Initial project files"
git remote add origin https://github.com/<username>/jenkins-demo.git
git push -u origin main
```

Create the empty repo jenkins-demo on GitHub first. Keep it public so Jenkins can clone without credentials.

b

Java 17 or 21 is required. Download jenkins.war from jenkins.io (or use the Windows installer), then:

```
java -version
java -jar jenkins.war --httpPort=8080
```

Open http://localhost:8080 and unlock with the password printed in the console (also in ~/.jenkins/secrets/initialAdminPassword; installer: C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword). Choose Install suggested plugins, then create the admin user.

c

Manage Jenkins > Plugins > Available plugins > search and install Git, GitHub, Pipeline > tick "Restart Jenkins when installation is complete". (Suggested plugins usually include them - check the Installed tab.)

d

Dashboard > New Item > name: ai-pipeline > select Pipeline > OK.

e

*Configure > Pipeline section > Definition: Pipeline script from SCM > SCM: Git > Repository URL: https://github.com/<username>/jenkins-demo.git > Branch Specifier: */main > Script Path: Jenkinsfile > Save.*

f

```
cat > Jenkinsfile << 'EOF'
pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps { checkout scm }
        }
        stage('Build') {
            steps { bat 'python --version' }
        }
        stage('Test') {
            steps { bat 'python app.py' }
        }
    }
    post {
        success { echo 'Build successful' }
        failure { echo 'Build failed' }
    }
}
EOF
git add Jenkinsfile
git commit -m "Add Jenkinsfile"
git push origin main
```

On Linux/macOS replace bat with sh.

g

*Configure > Build Triggers > tick Poll SCM > Schedule: * * * * * (every minute) > Save.*

	Webhook alternative: tick "GitHub hook trigger for GITScm polling", then GitHub repo > Settings > Webhooks > Payload URL http://<public-url>/github-webhook/ (needs a public URL such as ngrok because GitHub cannot reach localhost). Poll SCM is simpler for the exam.

h	*Click Build Now once so Jenkins records the first build, then push a change:*
	`echo "print('Updated version')" >> app.py`
	`git add .`
	`git commit -m "Update app.py to trigger Jenkins"`
	`git push origin main`

i	*Within a minute a new build (#2) appears in Build History. Open it: the cause says "Started by an SCM change"; Console Output ends with "Finished: SUCCESS". Screenshot the Stage View.*

Problem 9 - Docker: containerise an AI model

a	`mkdir ai-docker-app`
	`cd ai-docker-app`
	`cat > requirements.txt << 'EOF'`
	`flask`
	`scikit-learn`
	`EOF`
	`cat > app.py << 'EOF'`
	`from flask import Flask, request, jsonify`
	`from sklearn.datasets import load_iris`
	`from sklearn.linear_model import LogisticRegression`
	
	`app = Flask(__name__)`
	`X, y = load_iris(return_X_y=True)`
	`model = LogisticRegression(max_iter=200).fit(X, y)`
	
	`@app.route("/")`
	`def home():`
	`  return "AI Model API is running"`
	
	`@app.route("/predict", methods=["POST"])`
	`def predict():`
	`  features = request.json["features"]`
	`  pred = int(model.predict([features])[0])`
	`  return jsonify({"prediction": pred})`
	
	`if __name__ == "__main__":`
	`  app.run(host="0.0.0.0", port=5000)`
	`EOF`

b	`touch Dockerfile`

c	`cat > Dockerfile << 'EOF'`
	`FROM python:3.10-slim`
	`WORKDIR /app`
	`COPY requirements.txt .`
	`RUN pip install --no-cache-dir -r requirements.txt`
	`COPY . .`
	`EXPOSE 5000`
	`CMD ["python", "app.py"]`
	`EOF`

d	`docker version`
	`docker build -t ai-model-app .`
	`docker images`

e	`docker run -d -p 5000:5000 --name ai-container ai-model-app`
f	`docker ps`

	`docker logs ai-container`

g	`curl http://localhost:5000`
	`curl -X POST -H "Content-Type: application/json" -d '{"features": [5.1, 3.5, 1.4, 0.2]}' http://localhost:5000/predict`
	Or open http://localhost:5000 in the browser. Expected: {"prediction": 0}

h	`docker stop ai-container`
	`docker ps -a`
	`docker rm ai-container`
	If you see "port is already allocated", run with -p 5001:5000 and test on localhost:5001.

Problem 10 - Docker Compose: AI model + database + frontend

1 Folders	`mkdir -p ai-compose/backend ai-compose/frontend` `cd ai-compose`

2 Backend	`cat > backend/requirements.txt << 'EOF'`
	`flask`
	`flask-cors`
	`scikit-learn`
	`psycopg2-binary`
	`EOF`
	`cat > backend/app.py << 'EOF'`
	`import os, time, psycopg2`
	`from flask import Flask, request, jsonify`
	`from flask_cors import CORS`
	`from sklearn.datasets import load_iris`
	`from sklearn.linear_model import LogisticRegression`
	
	`app = Flask(__name__)`
	`CORS(app)`
	`X, y = load_iris(return_X_y=True)`
	`model = LogisticRegression(max_iter=200).fit(X, y)`
	
	`def get_conn():`
	`  return psycopg2.connect(host=os.getenv("DB_HOST", "db"), dbname=os.getenv("DB_NAME", "aidb"),`
	`  user=os.getenv("DB_USER", "postgres"), password=os.getenv("DB_PASS", "password"))`
	
	`def init_db():`
	`  for _ in range(10):`
	`  try:`
	`  conn = get_conn()`
	`  cur = conn.cursor()`
	`  cur.execute("CREATE TABLE IF NOT EXISTS predictions (id SERIAL PRIMARY KEY, features TEXT, result INT)")`
	`  conn.commit()`
	`  conn.close()`
	`  return`
	`  except Exception:`
	`  time.sleep(3)`
	
	`@app.route("/predict", methods=["POST"])`
	`def predict():`
	`  f = request.json["features"]`
	`  r = int(model.predict([f])[0])`
	`  conn = get_conn()`
	`  cur = conn.cursor()`
	`  cur.execute("INSERT INTO predictions (features, result) VALUES (%s, %s)", (str(f), r))`
	`  conn.commit()`
	`  conn.close()`
	`  return jsonify({"prediction": r})`

	``
	init_db()
	if __name__ == "__main__":
	    app.run(host="0.0.0.0", port=5000)
	EOF
	cat > backend/Dockerfile << 'EOF'
	FROM python:3.10-slim
	WORKDIR /app
	COPY requirements.txt .
	RUN pip install --no-cache-dir -r requirements.txt
	COPY . .
	EXPOSE 5000
	CMD ["python", "app.py"]
	EOF
	``

3 Frontend	```
	cat > frontend/index.html << 'EOF'
	<!DOCTYPE html>
	<html><body>
	<h2>Iris AI Predictor</h2>
	<input id="f" value="5.1,3.5,1.4,0.2">
	<button onclick="go()">Predict</button>
	<p id="out"></p>
	<script>
	async function go() {
	  const features = document.getElementById("f").value.split(",").map(Number);
	  const r = await fetch("http://localhost:5000/predict", {
	    method: "POST", headers: {"Content-Type": "application/json"},
	    body: JSON.stringify({features})
	  });
	  const d = await r.json();
	  document.getElementById("out").innerText = "Prediction class: " +
	d.prediction;
	}
	</script></body></html>
	EOF
	cat > frontend/Dockerfile << 'EOF'
	FROM nginx:alpine
	COPY index.html /usr/share/nginx/html/index.html
	EOF
	``

4 Compose	``` cat > docker-compose.yml << 'EOF' services: db: image: postgres:15 environment: POSTGRES_DB: aidb POSTGRES_USER: postgres POSTGRES_PASSWORD: password volumes: - pgdata:/var/lib/postgresql/data backend: build: ./backend ports: - "5000:5000" environment: DB_HOST: db DB_NAME: aidb DB_USER: postgres DB_PASS: password depends_on: - db frontend: build: ./frontend ports: - "3000:80" depends_on: - backend volumes: pgdata: EOF ```

5 Run	`docker compose up -d --build`
	`docker compose ps`
	`docker compose logs backend`

6 Test	*Open http://localhost:3000, click Predict. Then show the row stored in the database container:* `docker compose exec db psql -U postgres -d aidb -c "SELECT * FROM predictions;"`
7 Stop	`docker compose down` *Frontend uses port 3000 (not 8080) so it does not clash with Jenkins. Add -v to docker compose down to also delete the database volume.*

Problem 11 - Minikube: deploy and scale the Dockerised AI app

1 Image	*Docker Desktop must be running. Build the image from Problem 9 if it does not exist:*
	`cd ai-docker-app`
	`docker build -t ai-model-app .`

2 Start	`minikube start --driver=docker`
	`kubectl get nodes`
	`minikube status`

3 Load image	*Minikube has its own Docker, so copy the local image into it:*
	`minikube image load ai-model-app:latest`
	`minikube image ls`

4 Manifests	`mkdir k8s`
	`cat > k8s/deployment.yaml << 'EOF'`
	`apiVersion: apps/v1`
	`kind: Deployment`
	`metadata:`
	`  name: ai-deployment`
	`spec:`
	`  replicas: 2`
	`  selector:`
	`  matchLabels:`
	`  app: ai-app`
	`  template:`
	`  metadata:`
	`  labels:`
	`  app: ai-app`
	`  spec:`
	`  containers:`
	`  - name: ai-container`
	`  image: ai-model-app:latest`
	`  imagePullPolicy: Never`
	`  ports:`
	`  - containerPort: 5000`
	`EOF`
	`cat > k8s/service.yaml << 'EOF'`
	`apiVersion: v1`
	`kind: Service`
	`metadata:`
	`  name: ai-service`
	`spec:`
	`  type: NodePort`
	`  selector:`
	`  app: ai-app`
	`  ports:`
	`  - port: 5000`
	`  targetPort: 5000`
	`  nodePort: 30007`
	`EOF`

5 Deploy	`kubectl apply -f k8s/deployment.yaml`
	`kubectl apply -f k8s/service.yaml`
	`kubectl get deployments`
	`kubectl get pods`

	`kubectl get svc`

6 Access	`minikube service ai-service --url`
	Keep that terminal open (Docker driver), then in a new terminal:
	`curl <url-printed-above>`
	Alternative:
	`kubectl port-forward service/ai-service 5000:5000`
	`curl http://localhost:5000`

7 Scale	`kubectl scale deployment ai-deployment --replicas=4`
	`kubectl get pods -o wide`
	`kubectl get deployment ai-deployment`
	`kubectl scale deployment ai-deployment --replicas=2`
	Optional auto-scaling: minikube addons enable metrics-server, then kubectl autoscale deployment ai-deployment --cpu-percent=50 --min=2 --max=5 (needs CPU requests in the YAML).

8 Clean up	`kubectl delete -f k8s/` `minikube stop`

Problem 12 - CI/CD: Jenkins + Docker + Kubernetes (Minikube)

1 Project

Reuse app.py, requirements.txt and Dockerfile from Problem 9, and the k8s folder from Problem 11:

```
mkdir ai-cicd
cd ai-cicd
cp ../ai-docker-app/app.py ../ai-docker-app/requirements.txt
../ai-docker-app/Dockerfile .
cp -r ../ai-docker-app/k8s .
ls
```

2 Jenkinsfile

```
cat > Jenkinsfile << 'EOF'
pipeline {
    agent any
    environment {
        IMAGE = 'ai-model-app'
    }
    stages {
        stage('Checkout') {
            steps { checkout scm }
        }
        stage('Build Docker Image') {
            steps { bat "docker build -t ${IMAGE}:${BUILD_NUMBER} ." }
        }
        stage('Load Image into Minikube') {
            steps { bat "minikube image load ${IMAGE}:${BUILD_NUMBER}" }
        }
        stage('Deploy to Kubernetes') {
            steps {
                bat "kubectl apply -f k8s/deployment.yaml"
                bat "kubectl apply -f k8s/service.yaml"
                bat "kubectl set image deployment/ai-deployment ai-container=$
{IMAGE}:${BUILD_NUMBER}"
            }
        }
        stage('Verify') {
            steps {
                bat "kubectl rollout status deployment/ai-deployment"
                bat "kubectl get pods"
            }
        }
    }
    post {
        success { echo 'Deployment successful' }
        failure { echo 'Pipeline failed' }
    }
}
```

	`EOF` *On Linux/macOS replace bat with sh.*

3 GitHub	*Create an empty repo ai-cicd on GitHub first*
	`git init`
	`git branch -M main`
	`git add .`
	`git commit -m "AI app with Dockerfile, k8s manifests and Jenkinsfile"`
	`git remote add origin https://github.com/<username>/ai-cicd.git`
	`git push -u origin main`

4 Run Jenkins correctly	*Jenkins must be able to run docker, minikube and kubectl as YOU. So start the cluster, stop any Jenkins Windows service, and start Jenkins from git-bash:*
	`minikube start --driver=docker`
	`docker version`
	`kubectl get nodes`
	`java -jar jenkins.war --httpPort=8080`

5 Jenkins job	*Same as Problem 8: New Item > ai-cicd-pipeline > Pipeline > Definition: Pipeline script from SCM > Git > URL https://github.com/<username>/ai-cicd.git > Branch */main > Script Path Jenkinsfile > Build Triggers: Poll SCM * * * * * > Save > Build Now.*

6 Trigger and verify	*Change the response message and push - the whole flow should run by itself:*
	`sed -i 's/AI Model API is running/AI Model API v2 - deployed by Jenkins/' app.py`
	`git add .`
	`git commit -m "Update message to trigger CI/CD"`
	`git push origin main`
	Watch the Stage View: Checkout > Build > Load > Deploy > Verify all green. Then confirm the new version is live:
	`kubectl get pods`
	`kubectl rollout history deployment/ai-deployment`
	`minikube service ai-service --url`
	`curl <url-printed-above>`

Quick Fixes for Common Errors

Fix 1	*fatal: remote origin already exists*
	`git remote remove origin`
	`git remote add origin https://github.com/<username>/<repo>.git`

Fix 2	*error: src refspec main does not match any (no commit yet, or branch is called master)*
	`git branch -M main`
	`git add .`
	`git commit -m "Initial commit"`
	`git push -u origin main`

Fix 3	*Push rejected (fetch first / non-fast-forward) because GitHub repo already has a README*
	`git pull origin main --allow-unrelated-histories`
	`git push origin main`

Fix 4	*Docker: "port is already allocated"*
	`docker ps`
	`docker stop <container-name>`
	or map a different host port, e.g. -p 5001:5000

Fix 5	*Jenkins: 'docker' / 'kubectl' is not recognized. Jenkins is running as a Windows service with a different PATH/user. Stop the service and start Jenkins with java -jar jenkins.war from git-bash (see Problem 12, step 4).*
Fix 6	*Pod shows ErrImageNeverPull / ImagePullBackOff: the image is not inside Minikube.*

	`minikube image load ai-model-app:latest` `kubectl rollout restart deployment/ai-deployment`

> **FINAL EXAM TIP: do the practical in the exact order of the lettered steps, keep terminal output visible in screenshots, and use the same repo/folder names that you type in your answers.**